Assignment 2 COMP 532: OpenAI Gym Environment

Group Members

1. "Balasubramaniam, Vijeth" V.Balasubramaniam@liverpool.ac.uk, Student ID: 201664207
2. "Nagaraja, Manoj" M.Nagaraja@liverpool.ac.uk, Student ID: 201667140
3. "Shankar, Pavithra" P.Shankar@liverpool.ac.uk, Student ID: 201633650
4. "Sriramaneni, Pooja" P.Sriramaneni@liverpool.ac.uk, Student ID: 201671298

Introduction

We decided to implement the Taxi_v3 OpenAI gym game using Q-learning. The Taxi_v3 game was introduced by OpenAI as a classic reinforcement learning problem. Taxi_v3's goal is to pick up a passenger at a specific location and drop them off at another location in a real-world environment. This game features a 5x5 grid-world with four coloured locations: Red (the pickup location for the passenger), Blue (the drop-off location), Yellow (an empty taxi location), and Green (the wall location). During the game, the taxi must maximize its reward while minimizing its steps. The purpose of this report is to explain the solution and analyse the results of the experiment.

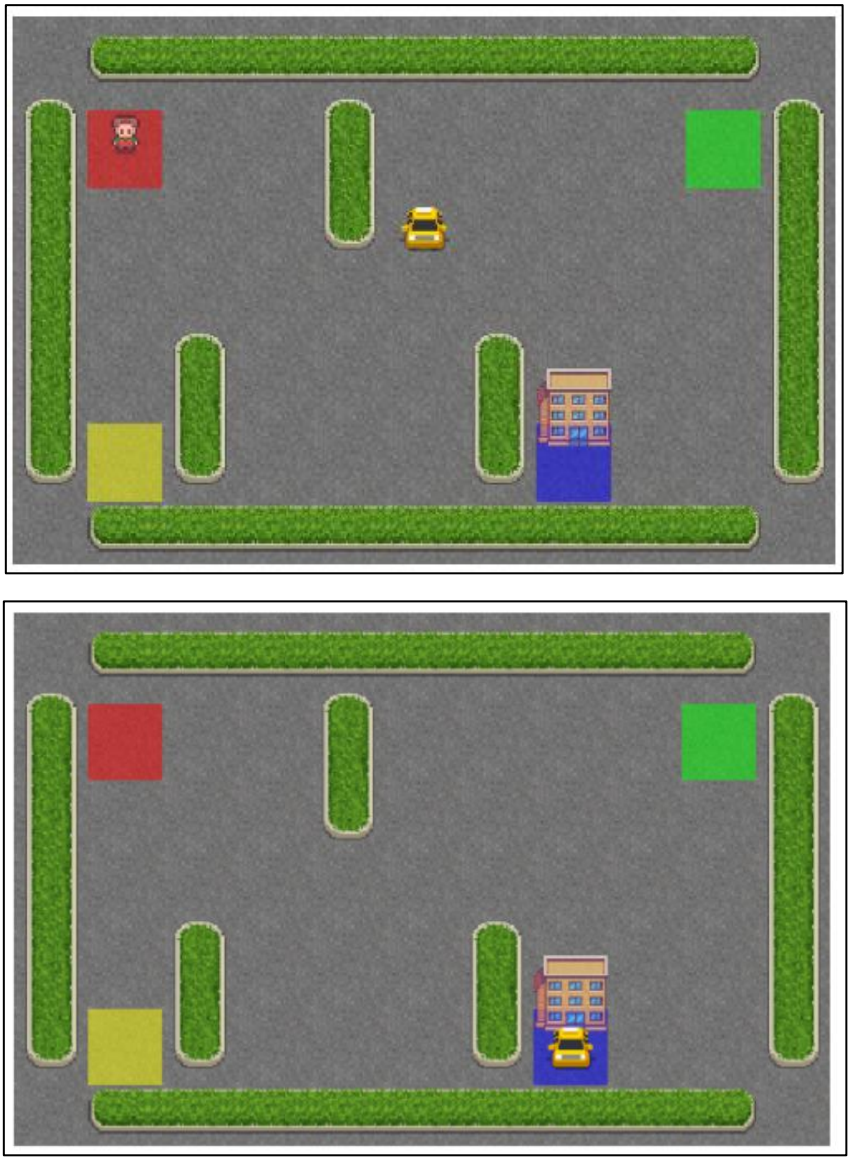


Figure 1: An Image of the Implemented Taxi_V3 Game

Methodology

In our python code, we imported the gym, random, numpy, Ipython, matplotlib, HTML and FunAnimation libraries. We then created the Taxi_V3 environment for use in our functions. Then, we employed the Q-learning algorithm, a reinforcement learning approach, to solve this game. The algorithm commences with zeros in the Q-table and iteratively updates the values when the taxi performs activities and earns rewards. The algorithm employs an exploration-exploitation trade-off to balance the exploitation of previously picked-up information with the investigation of new tasks. This algorithm learns an action-value function that gives each action in each state a value.

Discussions

The taxi was successfully trained by the Q-learning algorithm to go across the grid environment and receive high rewards. Over 10,000 episodes, the algorithm was able to produce an average reward of 50. Hyperparameters such the learning rate, discount factor, and exploration rate were changed during training. The hyperparameters used in this implementation were:

- Number of episodes: 10,000
- Max steps per episode: 100
- Learning rate: 0.1
- Discount factor: 0.99
- Exploration rate: 1.0
- Max exploration rate: 1.0
- Min rate: 0.01
- Exploration decay rate: 0.001

Exploration rate was initially set to 1.0 and decayed exponentially over time. As the taxi became better at navigating the grid environment, the exploration pace gradually declined, leading to a reduction in exploration.

We observed that the taxi's rewards increased over time as it learned to navigate the grid world more efficiently, as evidenced by the rewards plotted during training. The best reward obtained was 191, which was achieved in one of the episodes.

The reward is modified based on whether the drop off was successful or not. A reward of 20 to 200 is awarded if the drop off is successful to its destination. Agents should be encouraged to prioritize completing successful drop offs over simply completing a step in the environment, since successful drop offs are much more valuable.

If the taxi drops off the customer at wrong location, the original reward of -1 is reduced to -100. As a result, agents should be strongly discouraged from making such mistakes as incorrect drop offs are significantly more expensive than simply failing to deliver. The agent can improve its performance over time by learning how to modify its reward.

To visualize the trained taxi navigating the grid world, a video was created. The taxi shown in the video illustrates picking up and dropping off people at predetermined spots while dodging obstacles and taking the fewest possible steps.

Results

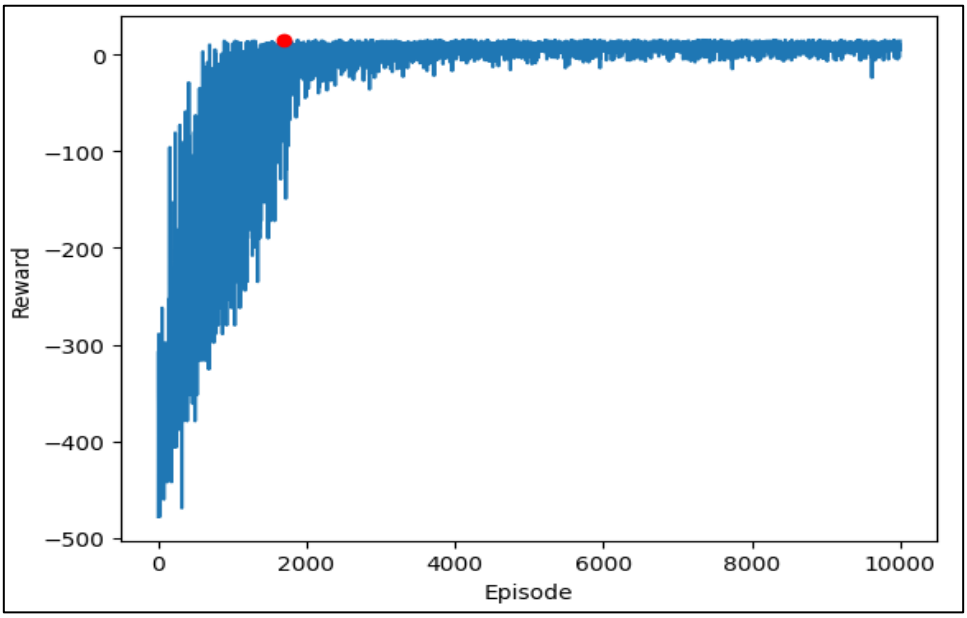


Figure 2: Number of Episodes and Rewards

The algorithm was able to play the game and get the best possible reward. A progressive increase in benefits over time is depicted in the storyline of the awards received in each episode. The best reward obtained is displayed in red on the plot. The animation displays an example game run in which the agent chooses actions at random.

```
Episode: 1000, Reward: 119, Avg. Reward: -142.7
Episode: 2000, Reward: 76, Avg. Reward: -43.0
Episode: 3000, Reward: 116, Avg. Reward: -2.6
Episode: 4000, Reward: 126, Avg. Reward: 62.2
Episode: 5000, Reward: 178, Avg. Reward: 109.7
Episode: 6000, Reward: -100, Avg. Reward: 126.9
Episode: 7000, Reward: 121, Avg. Reward: 148.0
Episode: 8000, Reward: 177, Avg. Reward: 179.0
Episode: 9000, Reward: 188, Avg. Reward: 178.3
Episode: 10000, Reward: 187, Avg. Reward: 186.5
Average reward: 50.0758
Success rate: 1.0
```

Figure 3: The output Screen

Conclusion

Future research might test the trained taxi in various grid worlds or settings to assess its generalizability. To evaluate which algorithm performs best on the Taxi-v3 game, additional reinforcement learning methods might be compared with the Q-learning algorithm. Overall, the Q-learning algorithm and the Taxi-v3 OpenAI game offer a great environment for investigating reinforcement learning and its uses in autonomous systems.

Question 2

Exploration and Exploitation for Deep Reinforcement Learning

The deep reinforcement learning ideas of exploration and exploitation are crucial to the Taxi v3 game. In this, the agent aims to discover the best possible strategy for maximising the expected cumulative reward. The agent must investigate its surroundings and take in the lessons it discovers in order to do this. Also, the agent must simultaneously operate in a manner that will most likely produce a high reward while also utilizing current information.

In the context of the Taxi-v3 game in exploration, an agent in the Taxi-v3 game explores different actions to learn about the environment rather than relying solely on its current Information. Based on the current Q-table estimate, actions are selected randomly from the action space or are suboptimal based on the current Q-table estimate. In order to avoid being trapped in suboptimal policy, the exploration phase is crucial. For instance, the agent can choose to go in a random path or pick up a passenger at a random place.

In contrast, exploitation involves the agent taking actions that are likely to result in high rewards based on its current information. A Taxi-v3 player selects an action that has the highest Q-table, which represents the expected reward associated with the action in its current condition. Agents must follow the strategy they have learned so far in the exploitation phase in order to maximize their profits. For instance, the agent will decide to take that course of action based on its existing information if it knows that picking up a passenger at one site and letting them off at another will result in a high reward.

An agent's learning process depends on a balance between exploration and exploitation. A high exploration rate can cause the agent to converge slowly, while a high exploitation rate can cause the agent to become stuck in a suboptimal policy. It is therefore important to gradually reduce exploration rates as an agent gains experience with its environment.